Random Forests for Network Intrusion Classification on UNSW-NB15: A Comparison with a Single Decision Tree and Nearest Neighbours

26913348
26913348@sun.ac.za

***Abstract*—This report investigates whether a random forest improves on a single decision tree and on a k-nearest neighbours classifier for classifying network traffic into ten classes in a modified UNSW-NB15 dataset. A random forest was chosen because the earlier decision tree suffered mainly from variance, and a forest of the same trees isolates the effect of ensembling. All models were implemented from scratch. The forest was tuned with a 120-trial tree-structured Parzen estimator search, followed by five targeted experiments. Between 20 and 50 trees were sufficient, random feature selection added little over bagging, and the leaf size controlled the trade-off between macro-F1 and balanced accuracy. Under five-fold stratified cross-validation, the forest achieved a balanced accuracy of 0.684 and a macro-F1 of 0.562, compared with 0.661 and 0.566 for the decision tree and 0.663 and 0.580 for k-nearest neighbours. Over three repetitions of the cross-validation, the balanced accuracy gain over the decision tree remained significant after correction for overlapping training folds and multiple comparisons, and the forest reduced false alarms on normal traffic. The large weighted leaves selected by the search, however, gave no gain in macro-F1. A follow-up experiment on the full dataset showed that a leaf size of ten raised the macro-F1 of the forest to 0.583 at a balanced accuracy of 0.668, above the decision tree on both metrics.**

I. Introduction

Network intrusion detection systems monitor network traffic and raise an alarm when a flow looks like an attack. Machine learning classifiers learn the difference between normal and malicious traffic from labelled examples, and can assign each flow to a specific attack category. The UNSW-NB15 dataset was created for this purpose and contains normal traffic together with nine categories of modern attacks [19]. Two properties make the classification difficult. The classes are strongly imbalanced, with the rarest attack category represented by fewer than 200 flows, and several attack categories produce very similar traffic statistics.

The decision tree and k-nearest neighbours (kNN) models developed in Assignment 2 were compared on a modified version of the dataset. The decision tree performed better, but overfitted when allowed to grow deep: its training score kept rising while its validation score fell. Ensemble methods combine many classifiers to reduce this kind of error [9]. This study tests whether an ensemble of trees improves on the single tree, and at what computational cost. Beyond the final scores, the experiments examine how the hyperparameters of the ensemble affect its behaviour on imbalanced classes.

This report selects an ensemble method that suits the weakness of the earlier decision tree, implements the ensemble and the earlier models without a machine learning library, tunes the ensemble to answer specific questions about the behaviour of the ensemble rather than only to maximise a score, and compares the tuned ensemble with the earlier models overall and per class.

The tuned random forest achieved a significantly higher balanced accuracy than the single tree and fewer false alarms on normal traffic, but did not improve macro-F1. The minimum leaf size largely decided the balance between the two metrics, and further testing on the full dataset showed that smaller leaves let the forest outperform the single tree on both. The remaining sections cover the background, methodology, implementation, experiments, results and conclusions.

II. Background

A. Ensemble Learning

An ensemble is a set of classifiers whose individual predictions are combined into a single prediction [9]. An ensemble improves on the individual members only when the members are accurate and make different errors [9]. If all members make the same errors, combining the members adds nothing.

The effect of combining members is clearest for averaging. Consider T identically distributed predictions, each with variance σ_h^2, and let ρ denote the correlation between any two of the predictions. The variance of the average of the T predictions is [14]

$$\rho\sigma_h^2 + \frac{1-\rho}{T}\sigma_h^2. \tag{1}$$

The second term vanishes as T grows, but the first term remains. Adding members therefore reduces variance only up to a floor set by the correlation between members. Reducing ρ lowers the floor itself. Averaging does not change the expected value of the predictions, so averaging leaves the bias of the members unchanged [14].

Ensemble methods differ in how the members are made different. Bagging trains each member on a different bootstrap sample of the training data and combines the members by voting or averaging [3]. Boosting trains the members sequentially, with each new member focusing on the instances that the current ensemble predicts poorly [11], [12]. Stacking trains a second-level model to combine the predictions of different base models [26]. Bagging mainly reduces variance, while boosting mainly reduces bias [14]. The choice of ensemble

method should therefore follow from whether the base model suffers more from variance or from bias.

B. Decision Trees

A decision tree classifies an instance by passing the instance down a sequence of tests on individual descriptive features until the instance reaches a leaf [5], [23]. The leaf assigns a class, usually the majority class of the training instances in that leaf. The class proportions in the leaf also give an estimate of the class probabilities.

A tree is grown top-down. At each node, the algorithm evaluates candidate splits and selects the split that most reduces node impurity. Let C denote the number of classes, and let q_c denote the proportion of the training instances at a node that belong to class c. Gini impurity is defined as

$$\text{Gini} = 1 - \sum_{c=1}^{C} q_c^2, \tag{2}$$

and entropy is defined as

$$\text{Entropy} = -\sum_{c=1}^{C} q_c \log_2 q_c. \tag{3}$$

Both measures are zero for a pure node and largest when the classes are evenly mixed. The quality of a split is the impurity of the parent node minus the weighted average impurity of the child nodes. For the entropy measure, this reduction is called the information gain [23].

Growth stops when a node is pure or when a stopping condition is reached. Common stopping conditions are a maximum depth and a minimum number of training instances per leaf. A tree grown without restrictions fits the training data almost perfectly. Small changes in the training data then lead to very different trees, which means that an unrestricted tree has low bias but high variance [14]. Restricting the depth or the leaf size reduces variance at the cost of extra bias.

Decision tree algorithms differ in how categorical features are handled. C4.5 creates one branch for every level of a categorical feature [23]. The classification and regression tree algorithm partitions the levels of a categorical feature into two groups [5]. Implementations that support only numeric thresholds require categorical features to be encoded first, for example with one binary indicator feature per level. The handling of categorical features is therefore a property of the implementation rather than of decision trees in general.

C. Bagging and Random Forests

Bagging, short for bootstrap aggregating, was introduced by Breiman [3]. Let D denote a training set of n instances. A bootstrap sample is drawn by selecting n instances from D uniformly at random with replacement [10]. A separate tree is grown on each of T bootstrap samples, and the T trees vote on the class of a new instance. The probability that a specific instance is excluded from a bootstrap sample is $(1 - 1/n)^n$, which approaches $e^{-1} \approx 0.368$ for large n. Each tree is therefore trained on about 63.2% of the distinct training instances. The remaining instances, called out-of-bag instances, provide a validation set for each tree at no extra cost [4].

Algorithm 1 Random forest training

for each tree index t from 1 to T **do**
  Draw a bootstrap sample D_t of n instances from D
  Create a root node containing all instances in D_t
  while a node can still be split under the stopping conditions **do**
    Select m of the p features uniformly at random
    Find the best split of the node over the m selected features
    Split the node into child nodes
  end while
  Store the grown tree as h_t
end for
Return the forest of trees h_1 to h_T

Bagged trees are grown from overlapping samples of the same data, so the trees remain strongly correlated. According to Equation (1), a high correlation limits the variance reduction that averaging can achieve. A random forest reduces the correlation further by also randomising the features [4]. Let p denote the number of descriptive features. At each node, only a random subset of $m \leq p$ features is considered when searching for the best split. A related idea, the random subspace method, selects one random feature subset for each whole tree rather than for each node [15].

The parameter m controls a trade-off. A small m makes the trees less correlated, but each tree is weaker because the best feature is often unavailable at a node. Breiman formalised this trade-off with two quantities [4]. The strength s is the expected margin of the forest, where the margin of an instance is the proportion of votes for the correct class minus the largest proportion of votes for any other class. Let $\bar{\rho}$ denote the mean correlation between the trees. Breiman showed that the generalisation error of the forest is bounded by

$$\frac{\bar{\rho}\,(1 - s^2)}{s^2}. \tag{4}$$

The bound decreases when the trees become stronger and when the trees become less correlated. The best value of m balances these two effects and depends on the dataset [22]. Common defaults are $m = \sqrt{p}$ for classification and $m = p/3$ for regression [14]. Setting $m = p$ reduces a random forest to bagging.

Algorithm 1 summarises the training procedure. Let h_t denote the tree trained on the t-th bootstrap sample, D_t. The original random forest combines the trees by majority vote [4]. An alternative is to average the class-probability estimates of the trees and to predict the class with the highest average. Breiman recommended growing the trees to full size without pruning, because averaging removes most of the variance of the individual trees [4]. Later studies showed that limiting the leaf size or depth can still help on noisy data [22].

Unlike most hyperparameters, the number of trees T does not need tuning in the usual sense. Increasing T does not cause overfitting, because the second term of Equation (1) only decreases [4]. Performance typically improves quickly over the first trees and then reaches a plateau [21]. The value of T therefore only needs to be large enough to reach the plateau, at a cost in training and prediction time that grows linearly with T.

A random forest also measures the importance of each feature. The mean decrease in impurity (MDI) of a feature sums the impurity reductions of all splits on that feature, weighted by the proportion of instances reaching each split, and averages the sums over all trees [18]. The MDI is known to favour features with many possible split points over binary features [25], so MDI values should be compared with care.

D. k-Nearest Neighbours

The k-nearest neighbours (kNN) classifier is an instance-based classifier that stores the training data and classifies a new instance by a vote among the k training instances closest to the new instance [7]. Let $\mathbf{x}$ and $\mathbf{x}'$ denote two instances, and let x_i and x'_i denote the values of the i-th descriptive feature of the two instances. The Manhattan distance between the instances is

$$\sum_{i=1}^{p} |x_i - x'_i|. \tag{5}$$

With distance weighting, each neighbour votes with a weight inversely proportional to the distance of the neighbour. The number of neighbours and the distance measure also control how sensitive kNN is to noise and outliers. A larger k spreads the vote over more neighbours, so a single noisy or extreme training instance has less influence on a prediction [14]. The Manhattan distance adds absolute differences rather than squared differences, so one large difference in a single feature dominates the distance less than under the Euclidean distance. Because kNN depends directly on distances, kNN is sensitive to the scale of features, to irrelevant features and to the number of dimensions [14]. Unlike a tree ensemble, kNN has no training phase in which the data is summarised, so the cost of kNN is incurred at prediction time.

E. Class Imbalance in Tree Ensembles

On imbalanced data, impurity-based splits and majority votes favour the majority classes. Two common corrections are resampling the training data and weighting the classes [6]. Let n_c denote the number of training instances of class c. A balanced class weight

$$w_c = \frac{n}{C\, n_c} \tag{6}$$

gives every class the same total weight. With class weights, the proportions q_c in the impurity measures and in the leaf votes are computed from weighted counts rather than raw counts. Chen *et al.* studied a weighted random forest of this form and found that the weighted forest improved minority-class recall [6].

The weights act through two mechanisms: the choice of splits and the class predicted by a leaf. The second mechanism only applies to impure leaves. A pure leaf predicts the class of the leaf instances regardless of the weights. Class weights therefore have a stronger effect on trees with larger leaves than on fully grown trees, in which most leaves are pure.

F. Summary

Averaging reduces variance, but the correlation between ensemble members limits how much variance can be removed. Random forests lower this correlation by drawing a random feature subset at every split. More trees do not cause overfitting, and class weights change predictions only in leaves that contain more than one class.

III. Methodology

A. Dataset and Shared Pre-Processing

The dataset is a modified version of the UNSW-NB15 network traffic dataset [19]. The dataset contains 257,673 instances, 43 descriptive features and a target feature, `attack_cat`, with ten classes. The classes are Normal traffic and nine attack categories: Reconnaissance, Backdoor, DoS, Exploits, Analysis, Fuzzers, Worms, Shellcode and Generic.

The pre-processing steps that apply to every model were kept the same as for the earlier models, so that the comparison isolates the effect of the models. The `id` feature was removed because the feature only identifies rows. Missing values occurred only in `service`, in 55% of the instances. The missing values were kept as a separate level, because the UNSW-NB15 documentation defines a missing service as a rarely used service rather than lost information [19]. Five features were removed because each feature had a Pearson correlation magnitude above 0.98 with another feature: `ct_ftp_cmd`, `sloss`, `dloss`, `dwin` and `ct_srv_dst`. The binary feature `is_ftp_login` contained 26 invalid values of 2 or 4, which were recoded as 1.

A total of 95,749 instances, or 37% of the dataset, were exact duplicates of other instances. Duplicates would allow the same instance to appear in both a training fold and a validation fold, which inflates validation scores. Duplicates were also concentrated in the Generic and DoS classes, so duplicates distorted the class distribution. All duplicates were therefore removed, leaving 161,924 instances and 37 descriptive features. The target remained strongly imbalanced, with 85,426 Normal instances and only 171 Worms instances.

B. Choice of Ensemble Method

The aim of the study was to determine how much an ensemble gains over the earlier decision tree, and why. A method similar to the earlier models was therefore preferred over a dissimilar method. A random forest uses the same type of tree as the earlier models, so differences in performance can be attributed to the ensemble mechanism itself rather than to a different model family.

The earlier decision tree also provided direct evidence for the choice. The validation macro-F1 of the single tree peaked

at a depth of about 15 to 20, while the training macro-F1 kept rising to about 0.80. The gap indicates that the single tree suffered mainly from variance. Section II-A showed that bagging-based methods target variance, while boosting targets bias. A random forest is therefore the ensemble method that addresses the weakness of the earlier decision tree. Gradient boosting was not selected because boosting would change both the base model and the error component being reduced, which would make the comparison harder to interpret.

C. Random Forest Pre-Processing

The forest predicts the class with the highest average class-probability estimate over the trees, and the trees of the forest used Gini impurity, whereas the earlier decision tree used entropy. The splitting criterion was not tuned, because Raileanu and Stoffel showed that Gini impurity and information gain select different splits in only about 2% of cases [24].

The trees described in Section IV split only on numeric thresholds, so `proto`, `service` and `state` were one-hot encoded. `service` and `state` have 13 and 11 levels respectively, so encoding the two features added few columns. `proto` has 133 levels, 127 of which occur fewer than 500 times. Encoding every `proto` level produces 191 encoded features on the full dataset, most of which are almost always zero. Grouping the levels that occur fewer than 500 times into a single level leaves five `proto` levels and reduces the total to 64 encoded features. For a single tree, sparse columns are harmless because the tree searches all columns at every node. For a random forest, sparse columns enter the random subset of m candidate features at every node. When m is small, rare `proto` columns can displace informative features from the candidate subset. Whether the rare levels should be grouped is therefore an open question for a random forest. Rather than fixing the encoding in advance, the study tested both options: every level encoded separately, and the rare levels grouped, as was done for the earlier kNN model.

Numeric features were neither scaled nor log-transformed. Tree splits depend only on the order of the values within a feature, so monotone transformations do not change the splits that a tree can make [14]. Outliers were kept for the same reason. Class imbalance was handled with class weights instead of resampling, so that no training instances were discarded. Two weighting schemes were tested. The `balanced` scheme computes the weights from the class counts of the whole training set. The `balanced_subsample` scheme recomputes the weights for each bootstrap sample.

D. Comparison Models

The earlier decision tree and kNN model were rebuilt with the implementation described in Section IV, using the same configurations. The decision tree used entropy, a maximum depth of 15, a minimum leaf size of two instances and balanced class weights. The tree used one-hot encoding of all levels of the three categorical features and unscaled numeric features. The 133 `proto` levels were not grouped for the tree, so that the tree remained the configuration that had been tuned for the earlier models, and because sparse columns are harmless to a tree that searches every feature at every node, as explained in Section III-C. The kNN pipeline grouped `proto` levels with fewer than 500 occurrences, log-transformed the 26 numeric features with a skewness above two, and standardised all numeric features. The pipeline then undersampled the classes larger than the median class size to the median size, and kept the six features ranked highest by a shallow decision tree. The kNN model used 109 neighbours, distance weighting and Manhattan distance. The tree had been tuned with a grid search and the kNN model with the tree-structured Parzen estimator (TPE) of Optuna [1], [2].

E. Summary

The random forest was selected mainly because the earlier tree showed high variance, and using the same base classifier isolates the effect of combining trees. The forest uses one-hot encoding without scaling and class weights against the imbalance. Whether to group rare `proto` levels was left to the experiments.

IV. Implementation

All models, pre-processing steps, sampling procedures and metrics were implemented from scratch in Python with NumPy [13], without a machine learning library. The code is available in a Git repository[1].

A. Feature Binning

A direct implementation of a tree sorts the values of every candidate feature at every node, which is slow in Python for 130,000 training instances and hundreds of trees. The implementation instead used the histogram approach of LightGBM [17]. Before a tree or forest is grown, every feature is mapped once to at most 256 bins and stored as an unsigned 8-bit integer. A feature with at most 256 distinct values receives one bin per value, with bin edges halfway between neighbouring values, which reproduces the thresholds of an exact tree. A feature with more distinct values is cut at 255 quantiles of the training data. All one-hot features and most count features fall into the first case, so binning only approximates the thresholds of the continuous features, such as `sbytes` and `rate`. The forest bins the data once and shares the binned matrix between all trees.

B. Tree Induction

Trees were grown depth first and stored as flat arrays of split features, threshold bins, child indices and leaf class distributions. Every training instance carries a weight, which combines the class weight with the number of times the instance was drawn into the bootstrap sample. Instances with weight zero are dropped at the root. The minimum leaf size counts instances, while the impurity and the leaf distributions use weights.

[1]Code: https://github.com/mattbennett04/DataScience/tree/main/assignment4

Features that are constant within the node are removed first, and the m candidate features are drawn from the remaining features. Two strategies were used to evaluate the candidates. For a node with at least 256 instances, a single weighted count over the combined index of feature, bin and class produces the class histogram of every candidate feature at once. Cumulative sums over the bins then give the impurity of every threshold of every feature in one vectorised step. For a node with fewer than 256 instances, filling 256 bins per feature costs more than sorting the few instances, so the instances are sorted by bin instead and the cumulative sums run over the sorted instances. Both strategies return the same split.

A node becomes a leaf when the node is pure, when the maximum depth is reached, when the node holds fewer instances than twice the minimum leaf size, or when no split decreases the impurity. The impurity decrease of every split is added to the importance of the split feature, which gives the MDI after normalisation. Prediction moves all instances down the tree together, one level per step, so the cost of prediction grows with the depth of the tree rather than with the number of instances.

C. Random Forest

A bootstrap sample was represented by the number of times each instance was drawn, which was then multiplied into the instance weights. No data is copied, and the out-of-bag instances are simply the instances with a count of zero. For the `balanced_subsample` scheme, the class weights are computed from the bootstrap counts of each tree.

Trees were grown in parallel in 16 worker processes. The binned data was placed in memory before the worker processes were started, so the workers shared the data instead of receiving copies. Each tree received its own random seed, derived from a single seed of the forest. The forest is therefore identical regardless of the number of worker processes, which kept every experiment reproducible.

D. k-Nearest Neighbours

The kNN classifier computes distances by brute force. The validation instances are processed in chunks of 256, so the distance matrix never exceeds 256 rows, and the chunks are distributed over 16 worker processes. The Manhattan distance is accumulated one feature at a time. For each validation instance, a partial sort selects the k nearest training instances without sorting all distances. With distance weighting, a training instance at distance zero receives all the weight. The encoder, scaler, undersampler and feature selector of the kNN pipeline were also implemented by hand, with the feature selector keeping the six features with the highest MDI in a tree of depth eight.

E. Verification

Each component was checked on small problems with known answers before the experiments were run, such as hand-computed impurities, the exclusive-or problem, agreement between the two split strategies, and kNN predictions from a direct loop over all training instances. The decision tree was also compared with a widely used reference implementation on the same fold and encoding. The macro-F1 of the two trees differed by 0.004. Training the reference tree on the binned features reduced the difference to 0.002, so binning explained about half of the gap, and the rest came from how ties between equally good splits are broken.

F. Summary

Binned features reduce the cost of finding splits, and the split search adapts to the number of instances in a node. Bootstrap samples are instance weights rather than copies, and trees are grown in parallel with fixed seeds. The implementation was checked on known cases and against a reference implementation.

V. Empirical Process

A. Research Questions

The tuning was designed to answer five questions, stated before any experiment was run:

1) How many trees are needed before the performance of the forest reaches a plateau?
2) How does the number of features considered per split, m, trade the strength of individual trees against the diversity of the trees, and where does the best balance lie for this dataset?
3) Does class weighting still trade macro-F1 against balanced accuracy in a forest, as observed for the single tree, and how does the effect of the weights depend on the leaf size?
4) Do fully grown trees overfit when the trees are averaged in a forest?
5) Does grouping the rare levels of `proto` help a random forest, and does the answer depend on m?

A final question concerned the comparison with the earlier models: does the tuned forest outperform the single decision tree and kNN, and on which classes?

B. Cross-Validation

All evaluations used five-fold stratified cross-validation. Stratification kept the class proportions equal across folds, which matters because Worms has only 171 instances, or about 34 per fold. The folds were produced by a stratified splitter that shuffles each class separately and deals the instances of the class out over the five folds. The final comparison used the same five folds for all three models. Each fold therefore produced one score for each model on identical training and validation data, which made the scores paired. All pre-processing steps that learn from data, namely the encoders, the scaler, the undersampler and the feature selector of kNN, were fitted inside each training fold only, to prevent information from the validation fold leaking into training.

C. Performance Metrics

Accuracy was not used, because a model that predicts mostly Normal traffic would achieve a high accuracy while missing most attacks. Instead, the models were evaluated with macro-F1 and balanced accuracy. Let TP_c, FP_c and FN_c denote the true positives, false positives and false negatives for class c. Precision and recall for class c are

$$\mathrm{precision}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}, \qquad \mathrm{recall}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}. \tag{7}$$

Precision is the proportion of predictions of class c that are correct, and recall is the proportion of class c instances that are found. The F1-score of class c is the harmonic mean of the two:

$$F1_c = \frac{2 \cdot \mathrm{precision}_c \cdot \mathrm{recall}_c}{\mathrm{precision}_c + \mathrm{recall}_c}. \tag{8}$$

Macro-F1 and balanced accuracy average the per-class F1-scores and recalls over all classes:

$$\text{macro-F1} = \frac{1}{C} \sum_{c=1}^{C} F1_c, \tag{9}$$

$$\text{balanced accuracy} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{recall}_c. \tag{10}$$

Every class contributes equally to both averages, so performance on the rare attack classes is not hidden by performance on Normal traffic. Balanced accuracy rewards finding the instances of every class, while macro-F1 also penalises false alarms.

D. Hyperparameter Search

The forest was tuned with Optuna [1], using the TPE sampler [2]. Optuna is a hyperparameter optimisation framework in which a study runs a sequence of trials, and each trial trains and scores one configuration chosen by a sampler. A grid search evaluates every combination of a fixed set of values, so the cost of a grid grows multiplicatively with each added hyperparameter. The forest had six hyperparameters, two of them continuous or integer-valued over wide ranges, so a grid fine enough to locate the optimum would have been too expensive. The TPE sampler instead splits the completed trials into a good group and a poor group by objective value, fits a density estimate to the hyperparameter values of each group, and proposes values that are likely under the good density and unlikely under the poor density [2]. The first 20 trials sampled values at random to give the density estimates a starting point.

Table I lists the search space. The range for m was expressed as a fraction of the encoded features, and covered the common default of $\sqrt{p}$, which corresponds to a fraction of about 0.07. The maximum depth was searched in two stages: a categorical choice of whether to limit the depth, followed by a depth between 8 and 40 when the depth was limited.

The objective of each trial was the mean, over the five folds, of the average of macro-F1 and balanced accuracy. Tuning the earlier decision tree showed that optimising macro-F1 alone selected a decision tree with similar macro-F1 but a balanced accuracy that was ten percentage points lower. Combining both metrics prevents the search from sacrificing one metric for the other. After the first ten trials, a median pruner stopped a trial after at least three folds if the running mean of the trial was below the median of earlier trials at the same fold. A total of 120 trials were run one after another, with each forest growing its trees in 16 parallel processes.

TABLE I
RANDOM FOREST SEARCH SPACE

Hyperparameter	Values searched
Fraction of features per split	$[0.05, 0.6]$
Class weighting	none, balanced, balanced subsample
Minimum leaf size	$[1, 50]$, log scale
Depth limited	yes, no
Maximum depth, if limited	$[8, 40]$
Rare `proto` levels grouped	yes, no

The search was run on a stratified sample of 50,000 instances, the same sample size that was used to tune the earlier decision tree. Training a forest on the full dataset for every trial would have taken several hours per configuration. Every trial used 100 trees. Fixing the number of trees during the search was justified because the number of trees does not cause overfitting, as discussed in Section II-C. The number of trees for the final model was chosen separately with the experiment described in the next subsection.

E. Targeted Experiments

The Optuna search finds a good configuration, but the trials do not vary one hyperparameter at a time, which makes the effect of each hyperparameter hard to isolate. Five targeted experiments were therefore run on the same 50,000-instance sample and folds, each varying one or two hyperparameters while the others were kept at the values selected by Optuna.

The ensemble-size experiment addressed the first question. A forest of 500 trees was grown in each fold. The prediction of a forest with the first T trees was obtained by averaging the class-probability estimates of those T trees, which is exactly the prediction a forest of T trees would make. Both metrics were recorded for T from 1 to 500.

The diversity experiment addressed the second question. With the selected encoding, the tuning sample had $p = 187$ encoded features. For each of nine values of m, namely the powers of two from 1 to 128 and $m = 187$, a forest of 100 trees was grown in each fold. Three quantities were recorded on the validation fold: the macro-F1 of the forest, the mean macro-F1 of the individual trees as a measure of strength, and the mean proportion of validation instances on which two trees agreed, averaged over 500 random pairs of trees, as a measure of correlation. The value $m = 187$ corresponds to bagging.

The weighting experiment addressed the third question. All three weighting schemes were combined with minimum leaf sizes of 1, 3, 10 and 30, with unlimited depth. The encoding experiment addressed the fifth question. Grouped and ungrouped `proto` encodings were combined with m set to $\sqrt{p}$ and to 15%, 30% and 50% of the features. The fourth question

was addressed with a validation curve over the maximum depth, from 2 to 60, for the tuned forest and for the earlier decision tree, recording both training and validation macro-F1.

F. Final Evaluation and Statistical Tests

The tuned forest, the decision tree and kNN were evaluated on the full dataset of 161,924 instances with the five folds described above. Predictions on each validation fold were collected to form one out-of-fold prediction for every instance, from which the per-class precision, recall and F1-score and the confusion matrix were computed. Unlike a single held-out split, the out-of-fold predictions use every instance exactly once for validation.

The forest was compared with each earlier model with paired t-tests on the five fold scores. Let δ_j denote the difference between the scores of two models on fold j, let $\bar{\delta}$ denote the mean of the differences over J folds, and let σ_δ^2 denote the sample variance of the differences. The standard paired t-statistic is

$$\tau = \frac{\bar{\delta}}{\sqrt{\sigma_\delta^2 / J}}. \tag{11}$$

The standard test assumes that the fold differences are independent. In cross-validation, the training sets of different folds overlap by 75%, so the differences are correlated and the standard test underestimates the variance [8]. Nadeau and Bengio proposed a corrected resampled t-test that accounts for the overlap [20]. Let ℓ and u denote the number of training and validation instances per fold. The corrected statistic is

$$\tau = \frac{\bar{\delta}}{\sqrt{\left(\frac{1}{J} + \frac{u}{\ell}\right) \sigma_\delta^2}}. \tag{12}$$

With five folds, $u/\ell = 0.25$, which more than doubles the variance term compared with the standard test. Both statistics were compared with a t-distribution with $J - 1$ degrees of freedom. The corrected test is the main test of the study, and the standard test is reported for comparability with the earlier models. Four comparisons were made, namely two models and two metrics, so the corrected p-values were also adjusted with the Holm procedure [16] to control the probability of any false positive at $\alpha = 0.05$. The Holm procedure sorts the p-values in increasing order and multiplies the i-th smallest p-value by the number of hypotheses not yet rejected.

Five folds give only four degrees of freedom, which limits the power of the corrected test. The whole comparison was therefore repeated with two further random assignments of the instances to five folds, which gave $J = 15$ paired scores per model. The corrected statistic applies to repeated cross-validation without change, with $J = 15$ and 14 degrees of freedom [20]. The repetitions shrink the $1/J$ term but leave the u/ℓ term, so the repeated test remains conservative.

The fit and prediction times of each model were also recorded on a machine with 16 processor cores. The forest and kNN used all 16 cores, while the decision tree used one core.

G. Follow-Up Analyses

Two further analyses checked the design choices of the study. First, the minimum leaf size had been tuned on training folds of 40,000 instances, but the final forest was trained on folds of about 130,000 instances. A minimum leaf size is an absolute number of instances, so the best value may change with the size of the training set. The forest was therefore also evaluated on the full dataset with leaf sizes of 3, 6 and 10, with all other hyperparameters unchanged and with the same five folds. A leaf size of 6 corresponds to the tuned value of 20 scaled by the ratio of the two training set sizes. This experiment used the final evaluation folds, so the experiment measures the sensitivity of the forest to the leaf size rather than selecting a new final model.

Second, the tuning sample was drawn from the same instances that were later used in the final cross-validation. Configurations that suit those 50,000 instances may score better on them than on new instances. The out-of-fold predictions were therefore also scored on only the 111,924 instances that were never part of the tuning sample.

H. Summary

Tuning used a 120-trial TPE search on a stratified sample of 50,000 instances, followed by five targeted experiments, one per research question. All three models were evaluated on the same folds of the full dataset, with corrected paired t-tests over three repetitions of five-fold cross-validation.

VI. Results and Discussion

A. Hyperparameter Search

Of the 120 trials, 104 ran to completion and 16 were pruned. The best objective rose from 0.525 in the first trial to 0.607 in trial 16 and 0.613 in trial 41. The remaining 79 trials improved the best value by less than 0.001, so the search converged well within the budget of 120 trials.

The selected configuration considered 44.3% of the 187 encoded features at each split, used balanced class weights, a minimum leaf size of 20 and unlimited depth, and encoded every `proto` level separately. On the tuning sample, the configuration achieved a macro-F1 of 0.545 ± 0.008 and a balanced accuracy of 0.682 ± 0.014. For reference, the earlier decision tree achieved 0.529 and 0.601 on the same sample and folds.

The importance of each hyperparameter was estimated with the functional analysis of variance method that Optuna provides. The method fits a regression model that predicts the objective from the hyperparameters of the completed trials, and attributes the variance of the predicted objective to each hyperparameter. The minimum leaf size explained 78% of the variation in the objective, followed by class weighting at 20%. The feature fraction, the `proto` encoding and the depth limit together explained about 2%. The targeted experiments in the following subsections explain these importances.

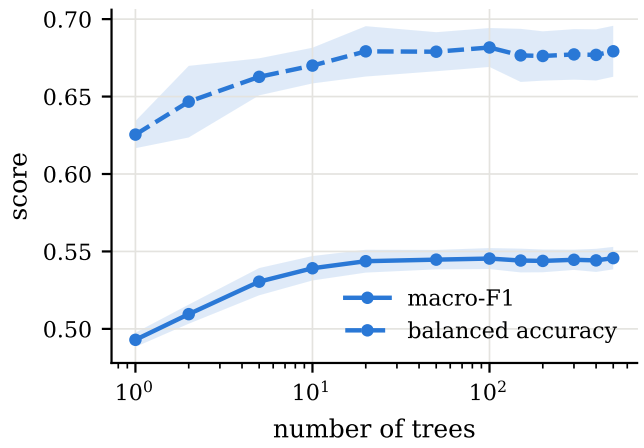


Fig. 1. Validation scores of the forest as a function of the number of trees

B. Number of Trees

Fig. 1 shows both metrics as a function of the number of trees. A single tree from the forest achieved a macro-F1 of 0.493 and a balanced accuracy of 0.625. Both metrics rose steeply up to about 20 trees. From 20 trees onwards, the macro-F1 stayed between 0.544 and 0.546, and the balanced accuracy between 0.676 and 0.682. These variations are smaller than the fold-to-fold standard deviations of 0.007 and 0.013. Neither metric decreased as trees were added, which agrees with the result that additional trees do not cause overfitting [4], [21].

On this dataset, 20 to 50 trees were enough to reach the plateau (Question 1). The final model used 200 trees, in case the larger training folds of the full dataset needed more trees to stabilise. The extra trees cost only training time.

C. Strength and Diversity of the Trees

Fig. 2 shows the results of the diversity experiment. As m increased from 1 to 187, the mean macro-F1 of the individual trees rose from 0.036 to 0.491, and the mean pairwise agreement rose from 0.17 to 0.79. Larger feature subsets therefore produced stronger but more similar trees, exactly the trade-off described by the bound in Equation (4).

The forest balanced the two effects. With $m = 1$, the individual trees were almost useless, yet the low agreement between the trees allowed the forest to reach a macro-F1 of 0.383. The forest reached a plateau of 0.545 for m between 16 and 32, and stayed within 0.002 of the plateau up to $m = 187$, which is bagging. Random feature selection therefore added almost nothing over bagging on this dataset. A likely reason is the concentration of the predictive information in a few features. When a small subset of features dominates, trees grown on all features still differ enough through their bootstrap samples, and restricting the features mainly weakens the trees.

The best balance between strength and diversity lay on a wide plateau from about 16 features upwards (Question 2). The selected fraction of 44.3%, or 82 features, falls inside this plateau. Because performance barely changed across the plateau, the feature fraction received only 1% of the importance in the search.

D. Class Weighting and Leaf Size

Fig. 3 shows both metrics for every combination of weighting scheme and leaf size, with unlimited depth. Without class weights, both metrics fell as the leaf size increased beyond three, from a macro-F1 of 0.538 and a balanced accuracy of 0.516 with three instances per leaf to 0.493 and 0.478 with 30 instances per leaf. Larger unweighted leaves predicted the majority class of the leaf more often, which hurt the rare classes.

With one instance per leaf, class weights made almost no difference: the balanced forest achieved 0.532 and 0.520, against 0.533 and 0.513 without weights. Fully grown trees end in pure leaves, and a pure leaf predicts the class of the leaf instances regardless of the weights. The weights could therefore act only through the choice of splits, as predicted in Section II-E. As soon as the leaves were allowed to contain several instances, the weights took effect. With three instances per leaf, the balanced forest achieved the highest macro-F1 of the experiment, 0.566, with a balanced accuracy of 0.597. Larger leaves raised the balanced accuracy further, to 0.649 at ten instances and 0.682 at 30 instances, but lowered the macro-F1 to 0.554 and 0.534.

As with the single tree, class weighting traded macro-F1 against balanced accuracy, but in the forest the leaf size controlled the trade-off (Question 3). Larger weighted leaves made the forest predict the rare classes more often, which found more rare-class instances at the cost of more false alarms. The search objective weighted both metrics equally, and along the curve in Fig. 3 the balanced accuracy rose faster than the macro-F1 fell, so the search selected a leaf size of 20, towards the balanced accuracy end. This also explains why leaf size and class weighting dominated the hyperparameter importances: once weights were used, the leaf size decided which metric the forest favoured. The two weighting schemes performed almost identically, with differences of at most 0.002.

E. Tree Depth and Overfitting

Fig. 4 compares the training and validation macro-F1 of the tuned forest and the earlier decision tree as functions of the maximum depth. The single tree reached the highest validation macro-F1, 0.529, at a depth of 15. Deeper trees continued to improve the training score, up to 0.841, while the validation score fell to 0.513 at a depth of 60. The gap between the training and validation scores of the single tree reached 0.33.

The forest showed a different pattern. The validation macro-F1 of the forest rose with depth until a depth of about 25 and then stayed at 0.545, with no decrease up to a depth of 60. The training score levelled off at 0.611, so the gap between training and validation was only 0.07. Part of the smaller gap came from the minimum leaf size of 20, which limited how closely each tree could fit the training data. The absence

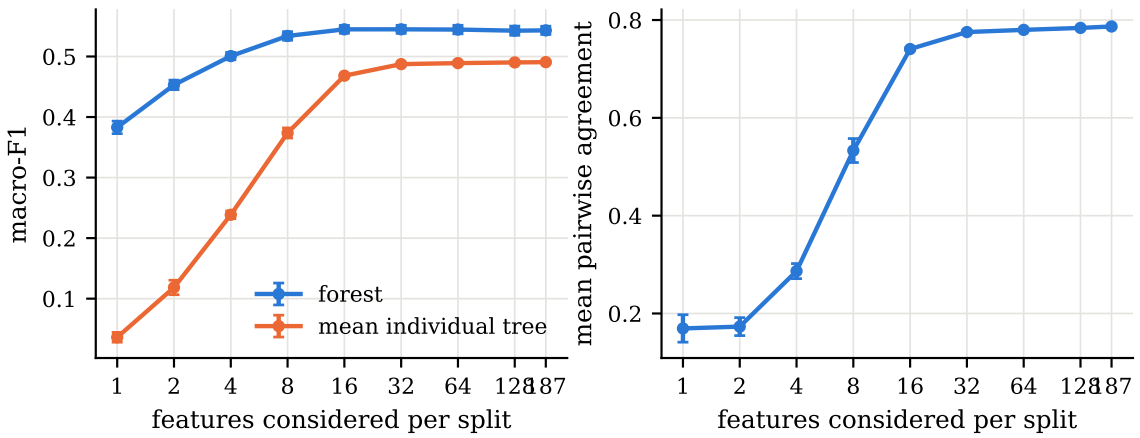


Fig. 2. Forest and individual tree macro-F1, and mean agreement between pairs of trees, as functions of the number of features considered per split

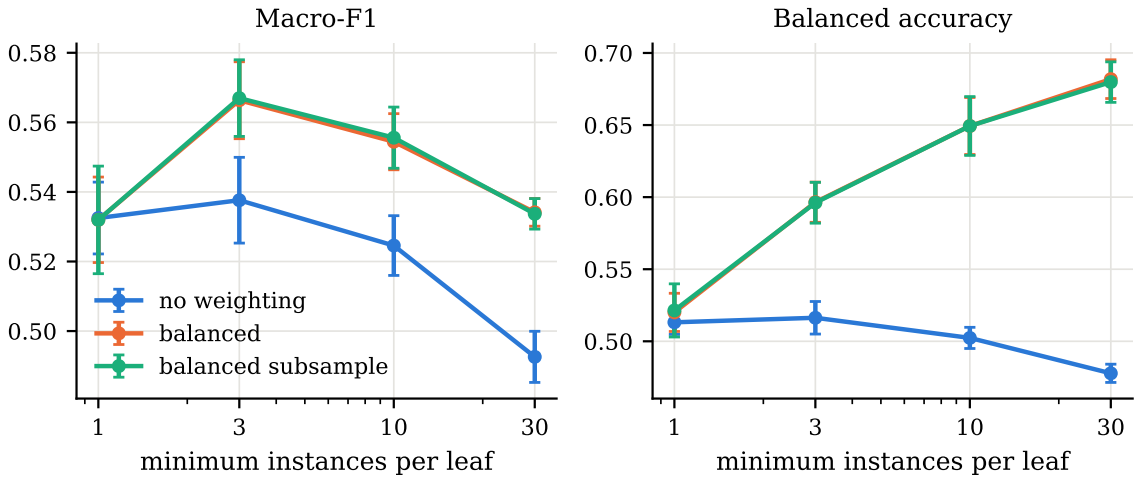


Fig. 3. Validation macro-F1 and balanced accuracy for each class weighting scheme as a function of the minimum leaf size

of any decline, however, shows that averaging removed the overfitting that deep single trees suffered from. Fully grown trees therefore did not overfit inside the forest (Question 4), which is why the depth limit received almost no importance in the search.

F. Encoding of Rare Protocol Levels

Table II compares the two `proto` encodings for four values of m. Contrary to the expectation in Section III-C, grouping the rare levels never improved either metric. With $m = \sqrt{p}$ and with 15% of the features, grouping lowered the macro-F1 by about 0.01. With 30% or 50% of the features, the difference was at most 0.002.

Two observations explain the result. First, the implementation removes features that are constant in a node before drawing the candidates, as described in Section IV-B. A rare `proto` column is zero for almost every instance, so the column becomes constant within a few splits of the root and stops competing for the candidate slots. The displacement effect that motivated the grouping therefore only applied near the root. Second, the rare levels carried information. Of the 5,900 instances with a `proto` level that occurs fewer than 500 times, 99.1% were attacks, compared with 47.2% of all instances. Grouping kept the fact that a protocol was rare, but discarded which rare protocol was used, which helps to tell attack categories apart.

Grouping therefore did not help a forest whose split search

TABLE II
VALIDATION SCORES WITH AND WITHOUT GROUPING OF RARE PROTOCOL LEVELS

	Macro-F1		Balanced accuracy	
Features per split	All levels	Grouped	All levels	Grouped
$\sqrt{p}$	0.543	0.535	0.672	0.664
15%	0.546	0.536	0.675	0.662
30%	0.545	0.546	0.675	0.675
50%	0.546	0.546	0.679	0.677

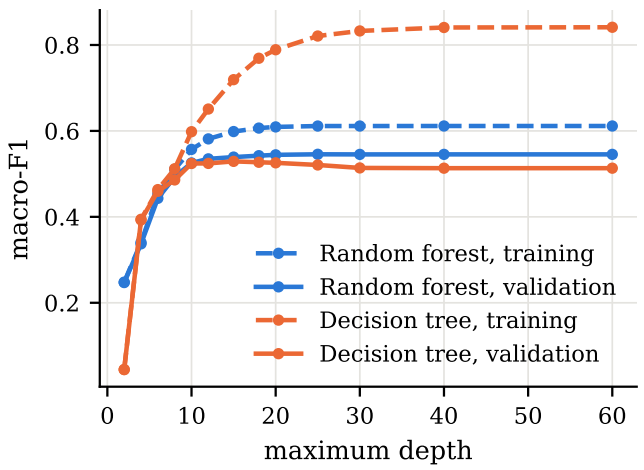


Fig. 4. Training and validation macro-F1 of the random forest and the decision tree as functions of the maximum depth

TABLE III
FIVE-FOLD CROSS-VALIDATION RESULTS ON THE FULL DATASET

Model	Macro-F1	Balanced accuracy	Fit time (s)
Random forest	0.562 ± 0.004	$\mathbf{0.684 \pm 0.004}$	99.6
Decision tree	0.566 ± 0.003	0.661 ± 0.003	3.8
kNN	$\mathbf{0.580 \pm 0.012}$	0.663 ± 0.016	0.7

ignores constant features, and slightly hurt when few features were considered per split (Question 5). The search accordingly kept the protocol levels separate.

G. Comparison with the Earlier Models

Table III shows the cross-validation results of the three models on the full dataset of 161,924 instances. The rebuilt tree and kNN model scored within 0.02 of their original scores on both metrics. The differences come from the folds, binning, the random draws of the undersampler and tie-breaking between equally good splits, and do not affect the comparison, because all three models used the same folds and implementation.

The random forest achieved the highest balanced accuracy, 0.684, but the lowest macro-F1, 0.562. On the single run of five folds, the balanced accuracy gain of 0.023 over the tree was significant under the standard test, $p < 0.001$, the corrected test, $p = 0.004$, and the Holm adjustment, $p = 0.015$. Against kNN, the standard test declared both differences significant, $p = 0.035$ for macro-F1 and $p = 0.050$ for balanced accuracy, but the corrected test did not, so the correction for overlapping training folds changed both conclusions.

Table IV shows the corrected tests over three repetitions of five-fold cross-validation, which are the main tests of the study. The mean scores barely changed: the forest achieved 0.563 ± 0.004 and 0.685 ± 0.007, the tree 0.566 ± 0.005 and 0.662 ± 0.005, and kNN 0.584 ± 0.010 and 0.667 ± 0.013. The balanced accuracy of the forest exceeded that of the tree in all 15 folds, and the gain remained significant after the Holm

TABLE IV
CORRECTED t-TESTS OVER THREE REPETITIONS OF FIVE-FOLD CROSS-VALIDATION

Versus	Metric	$\bar{\delta}$	Forest wins	Corrected p	Holm p
Tree	Macro-F1	-0.004	3 of 15	0.206	0.206
Tree	Bal. acc.	$+0.022$	15 of 15	<0.001	$\mathbf{< 0.001}$
kNN	Macro-F1	-0.022	0 of 15	0.006	**0.019**
kNN	Bal. acc.	$+0.018$	14 of 15	0.069	0.137

adjustment, $p < 0.001$. The extra repetitions also changed one conclusion. The macro-F1 of kNN exceeded that of the forest in all 15 folds, and with the added power the difference of 0.022 became significant, with a Holm-adjusted p-value of 0.019. The balanced accuracy gain of the forest over kNN remained not significant, $p = 0.137$, because kNN varied strongly between folds.

Scoring the out-of-fold predictions on only the 111,924 instances outside the tuning sample gave a macro-F1 of 0.564 and a balanced accuracy of 0.686 for the forest, against 0.562 and 0.684 on all instances. The tree and kNN changed by at most 0.002. The forest therefore did not score better on the instances used for tuning, and the overlap between the tuning sample and the evaluation data did not inflate the final scores.

Training the forest was considerably more expensive. Using 16 worker processes, the forest took 99.6 s per fold to train, compared with 3.8 s for the tree in a single process, which is roughly 400 times more processor time. Prediction took 1.3 s for the forest and 0.1 s for the tree for about 32,000 instances, while kNN required 9.3 s because kNN computes all distances at prediction time.

H. Per-Class Behaviour

Fig. 5 shows the F1-score of each class for the three models, computed from the out-of-fold predictions. Compared with the tree, the forest improved the F1-score on six classes: Normal, Reconnaissance, DoS, Exploits, Fuzzers and Generic, by between 0.01 and 0.04. These six classes are the six largest in the dataset, so enough data was available for averaging to stabilise the decision boundaries.

The improvement on Normal traffic matters in practice. The tree recalled 76.5% of Normal instances, while the forest recalled 81.1%. The forest therefore raised a false alarm on 18.9% of normal traffic, compared with 23.5% for the tree, mainly because fewer normal flows were labelled as Fuzzers. Fewer false alarms reduce the workload of analysts in an intrusion detection system.

The forest lost on the four smallest attack classes, and on Worms in particular. The forest recalled 88.3% of Worms instances, compared with 66.1% for the tree, but the precision fell from 0.45 to 0.25. The forest predicted Worms 602 times, of which only 151 were Worms and 234 were Exploits. The drop in the Worms F1-score alone, from 0.536 to 0.391, lowered the macro-F1 of the forest by 0.015, which is more than the total difference in macro-F1 between the forest and the tree. Large leaves with balanced weights give the 171

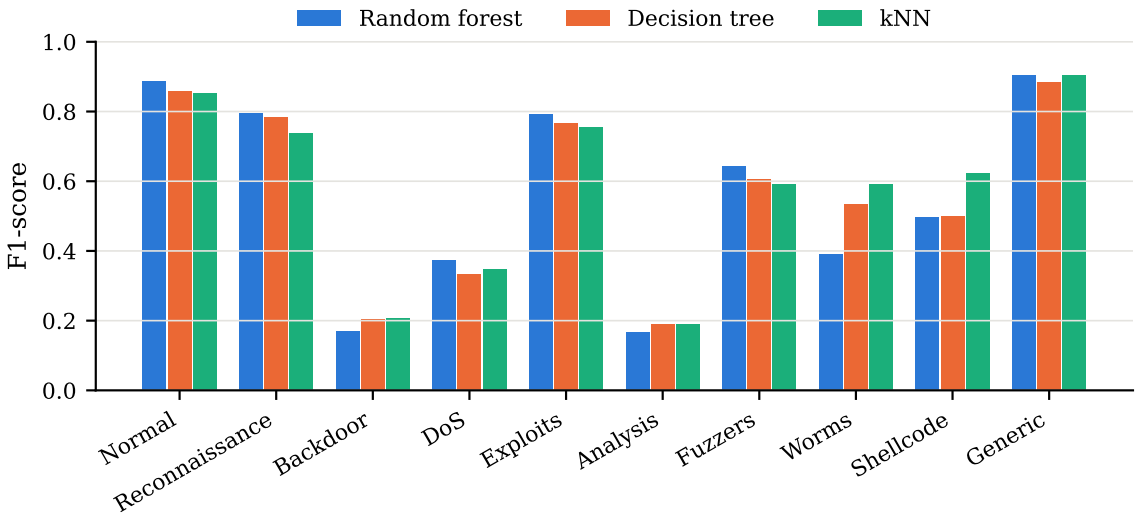


Fig. 5. Per-class F1-score of the three models, computed from the out-of-fold predictions

Worms instances a very large weight, so a leaf that contains a few Worms instances among many Exploits instances is still labelled Worms. The loss in macro-F1 was therefore the price of the high balanced accuracy, and followed directly from the leaf size selected by the search.

Fig. 6 shows the row-normalised confusion matrix of the forest. Backdoor and Analysis were almost indistinguishable: the forest labelled 60% of Backdoor instances as Analysis and 55% of Analysis instances as Backdoor. The tree confused the two classes as well, at 42% and 46%. A large overlap of this kind means that the two classes are not separable with the available features. A second overlap was between DoS and Exploits, with 21% of DoS instances predicted as Exploits by both models.

Both the forest and the tree relied most on `sbytes`, the number of bytes sent by the source, which received an MDI of about 0.21 in both models. The top 15 of the 187 features accounted for 85% of the total MDI in the forest and 89% in the tree. This concentration supports the explanation in Section VI-C for why random feature selection added little over bagging, although the MDI favours features with many split points [25].

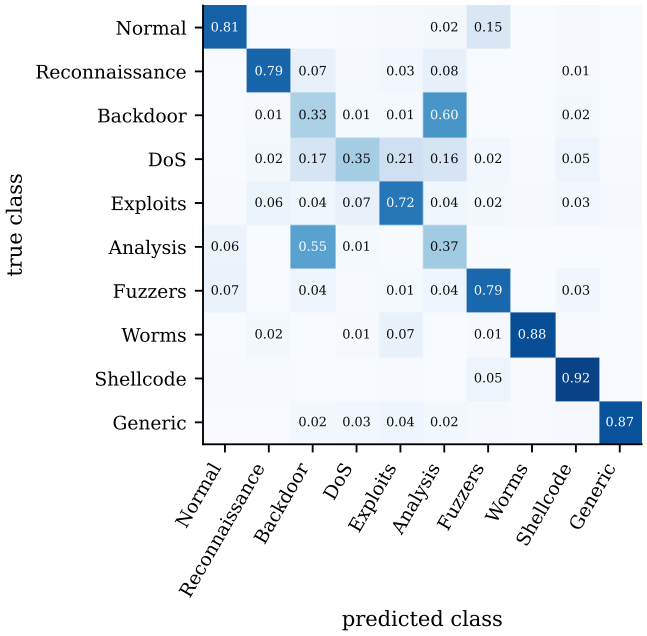


Fig. 6. Row-normalised confusion matrix of the random forest from the out-of-fold predictions

I. Leaf Size on the Full Dataset

Table V shows the forest on the full dataset with four leaf sizes, and Fig. 7 places the four forests and the two earlier models in the plane of the two metrics. Smaller leaves raised the macro-F1 and lowered the balanced accuracy, the same trade-off that the weighting experiment showed on the tuning sample. With a leaf size of 3 or 6, the forest achieved a macro-F1 of 0.593, which is 0.027 above the tree and 0.013 above kNN, but the balanced accuracy fell to 0.612 and 0.649.

With a leaf size of 10, the forest exceeded the tree on both metrics, with a macro-F1 of 0.583 against 0.566 and a balanced accuracy of 0.668 against 0.661. The macro-F1 gain was significant after the Holm adjustment, $p = 0.023$, while the balanced accuracy gain of 0.007 was not. The forest with a leaf size of 10 also matched kNN, at 0.580 and 0.663. Fig. 7 shows that both earlier models lie below the curve of the forest. The forest with a leaf size of 10 scored higher than both earlier models on both metrics.

The leaf size of 20 selected on the tuning sample was therefore too large for the full dataset. These results come from the final evaluation folds, so the leaf size of 10 should not be read as a tuned result. The results do, however, show

TABLE V
RANDOM FOREST ON THE FULL DATASET WITH DIFFERENT LEAF SIZES

Leaf size	Macro-F1	p vs tree	Bal. acc.	p vs tree
3	0.593 ± 0.007	**0.022**	0.612 ± 0.008	**0.005**
6	0.593 ± 0.004	**0.017**	0.649 ± 0.009	0.284
10	0.583 ± 0.003	**0.023**	0.668 ± 0.005	0.359
20	0.562 ± 0.004	0.375	0.684 ± 0.004	**0.022**

p-values from the corrected test, Holm-adjusted over the eight tests.

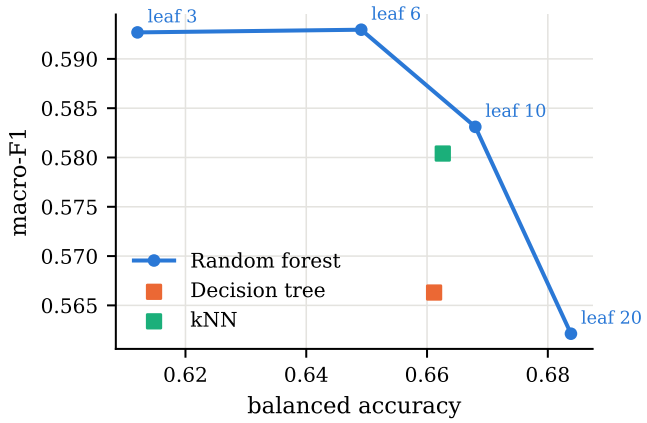


Fig. 7. Macro-F1 against balanced accuracy for the random forest with leaf sizes of 3, 6, 10 and 20, and for the decision tree and kNN, on the full dataset

that the macro-F1 deficit of the tuned forest was caused by the mismatch between the tuning sample and the full dataset, rather than by a limitation of the forest. A minimum leaf size is an absolute number of instances, so tuning on the full dataset, or expressing the leaf size as a fraction of the training set, would remove this mismatch at a higher computational cost.

J. Discussion

The tuning results agreed with the expectations from Section II, including the trade-off between strength and diversity described by Breiman [4], except that sparse `proto` columns did not harm the forest.

On the tuning sample, the forest offered a better trade-off curve than the tree, and the leaf size chose a point on that curve: with a leaf size of 3, the forest achieved 0.566 and 0.597 against 0.529 and 0.601 for the tree, and with a leaf size of 20 the forest exceeded the tree on both metrics. The equal weighting of the two metrics in the search objective placed the selected forest near the balanced accuracy end of the curve. A study that cares more about false alarms than about missed attacks should weight macro-F1 more heavily, which would select a smaller leaf.

The depth experiment showed that the forest removed most of the variance of the single tree: the gap between training and validation scores fell from 0.33 to 0.07. Equation (1) shows that averaging leaves the bias and the irreducible error unchanged. The remaining error came mostly from classes that overlap in the feature space, such as Backdoor and Analysis, and DoS and Exploits. No amount of averaging can separate classes whose feature values coincide, so further improvement would require more informative features rather than a stronger ensemble.

K. Summary

The tuned forest improved balanced accuracy over the single tree and reduced false alarms on normal traffic, but did not improve macro-F1. The cause was the minimum leaf size, which had been tuned on a smaller training set. With a leaf size of 10 on the full dataset, the forest exceeded the tree on both metrics.

VII. CONCLUSION

This study compared a random forest with the earlier decision tree and kNN models on ten-class network intrusion classification. The random forest was chosen because the earlier tree showed high variance, and because using the same base classifier isolated the effect of ensembling. All three models were implemented from scratch and evaluated on the same stratified folds.

Between 20 and 50 trees were enough for performance to stabilise, random feature selection added little over bagging, and fully grown trees did not overfit inside the forest. Once class weights were used, the minimum leaf size decided most of the trade-off between macro-F1 and balanced accuracy.

On the full dataset, the tuned forest achieved a balanced accuracy of 0.684, significantly higher than the decision tree under the corrected resampled t-test and Holm adjustment over three repetitions of cross-validation. It also reduced false alarms on normal traffic from 23.5% to 18.9%. Its macro-F1 of 0.562, however, was no better than that of the tree and significantly lower than that of kNN, mainly because the large weighted leaves produced many false Worms predictions. The leaf size of 20 had been selected on smaller training sets, and with a leaf size of 10 the forest achieved 0.583 and 0.668, above the tree on both metrics. Future work should weight the search objective by the cost of false alarms and missed attacks, tune the leaf size on training sets of the final size, and look for features that separate overlapping classes such as Backdoor and Analysis.

REFERENCES

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[2] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems*, vol. 24, 2011, pp. 2546–2554.

[3] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[6] C. Chen, A. Liaw, and L. Breiman, "Using random forest to learn imbalanced data," Department of Statistics, University of California, Berkeley, Technical Report 666, 2004.

[7] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[8] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[9] T. G. Dietterich, "Ensemble methods in machine learning," in *Proceedings of the First International Workshop on Multiple Classifier Systems*, 2000, pp. 1–15.

[10] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY: Chapman and Hall, 1993.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[13] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[14] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY: Springer, 2009.

[15] T. K. Ho, "The random subspace method for constructing decision forests," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 8, pp. 832–844, 1998.

[16] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[17] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

[18] G. Louppe, L. Wehenkel, A. Sutera, and P. Geurts, "Understanding variable importances in forests of randomized trees," in *Advances in Neural Information Processing Systems*, vol. 26, 2013, pp. 431–439.

[19] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proceedings of the Military Communications and Information Systems Conference*, 2015, pp. 1–6.

[20] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[21] P. Probst and A.-L. Boulesteix, "To tune or not to tune the number of trees in random forest," *Journal of Machine Learning Research*, vol. 18, no. 181, pp. 1–18, 2018.

[22] P. Probst, M. N. Wright, and A.-L. Boulesteix, "Hyperparameters and tuning strategies for random forest," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 9, no. 3, e1301, 2019.

[23] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993.

[24] L. E. Raileanu and K. Stoffel, "Theoretical comparison between the Gini index and information gain criteria," *Annals of Mathematics and Artificial Intelligence*, vol. 41, no. 1, pp. 77–93, 2004.

[25] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: Illustrations, sources and a solution," *BMC Bioinformatics*, vol. 8, no. 25, pp. 1–21, 2007.

[26] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

APPENDIX A
ACRONYMS

kNN	k-nearest neighbours
MDI	mean decrease in impurity
TPE	tree-structured Parzen estimator

APPENDIX B
SYMBOLS

α	significance level
C	number of classes
c	class index
D	training set
D_t	bootstrap sample of tree t
δ_j	score difference between two models on fold j
$\bar{\delta}$	mean of the score differences
FN_c	false negatives for class c
FP_c	false positives for class c
h_t	tree trained on D_t
i	feature index
J	number of folds
j	fold index
k	number of neighbours in kNN
ℓ	number of training instances per fold
m	number of features considered per split
n	number of training instances
n_c	number of training instances of class c
p	number of descriptive features
q_c	proportion of class c at a node
ρ	correlation between two predictions
$\bar{\rho}$	mean correlation between trees
s	strength of a forest
σ_h^2	variance of one prediction
σ_δ^2	sample variance of the score differences
T	number of trees
t	tree index
TP_c	true positives for class c
τ	paired t-statistic
u	number of validation instances per fold
w_c	weight of class c
$\mathbf{x}$, $\mathbf{x}'$	instances
x_i, x'_i	value of feature i of $\mathbf{x}$ and $\mathbf{x}'$